

Map

A estrutura de dados **map** permite armazenar uma coleção de itens, onde cada item consiste em duas informações: **chave** e **valor**. A **chave** é utilizada para identificar qual valor deve ser acessado ou modificado, enquanto o **valor** armazena a informação que estamos interessados.

A grande vantagem do **map** é que ele permite usar chaves de tipos diversos, diferente do vetor que podemos utilizar apenas índices inteiros para identificar os elementos. Por exemplo, podemos usar um mapa para associar nomes a idades:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    map<string, int> nome_idade;
    nome_idade["joao"] = 12;
    nome_idade["enzo"] = 20;
    nome_idade["francisco"] = 35;

    cout << nome_idade["joao"] << endl;
    // imprime: 12
}
```

Outra vantagem é que o **map** é uma estrutura dinâmica: a quantidade de memória utilizada é proporcional a quantidade de itens que se tem guardado. Não importa qual tamanho do índice, o que importa é a quantidade de elementos. Podemos utilizar isso para, por exemplo, usar índices inteiros que normalmente não caberiam em um vetor:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    map<int, int> mapa;
    mapa[1] = 2000;
    mapa[12345678] = 9;
    mapa[-1] = 3;

    cout << mapa.size() << endl;
    // imprime: 3
}
```

Declaração

Para declarar um mapa, você precisa especificar o tipo da chave e o tipo do valor que será guardado:

```
#include <bits/stdc++.h>
using namespace std;
```

```
int main(){
    // inicialmente vazio
    // mapeia int para string
    map<int, string> q;
}
```

Inserção e atualização

Você pode adicionar um elemento com `mapa[chave] = valor`. Se a chave já existir, o valor será atualizado. Caso contrário, um novo item é adicionado ao mapa:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    map<int, string> mapa;
    mapa[1] = "um";
    mapa[2] = "dois";
    mapa[3] = "três";

    // imprime: dois
    cout << mapa[2] << endl;
}
```

Busca

Você pode acessar valores salvos no map utilizando `mapa[chave]`. Se a chave existir, o valor será retornado. Caso contrário, um valor nulo (inteiro 0 ou string vazia) é inserido nessa chave e retornado:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    map<int, string> mapa;
    mapa[1] = "um";
    mapa[2] = "dois";
    mapa[3] = "três";

    // imprime: três
    cout << mapa[3] << endl;

    // imprime uma string vazia
    cout << mapa[4] << endl;
}
```

Alternativamente, podemos acessar um elemento usando `mapa.find(chave)`. Nesse caso, o elemento retorna um ponteiro para o elemento buscado. Caso não encontre a chave, é retornado um ponteiro para o final do mapa. O `find` pode ser utilizado para verificar se existe uma determinada chave na coleção:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    map<int, string> mapa;
    mapa[1] = "um";
    mapa[2] = "dois";
    mapa[3] = "três";

    if(mapa.find(4) != mapa.end()){
        cout << mapa[4] << endl;
    } else {
        cout << "Chave não encontrada!" << endl;
    }
}
```

No exemplo acima, a saída consiste em "Chave não encontrada".

Remoção

Para remover um elemento, podemos utilizar o método `mapa.erase(chave)`:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    map<int, string> mapa;
    mapa[1] = "um";
    mapa[2] = "dois";
    mapa[3] = "três";

    mapa.erase(2);

    if(mapa.find(2) != mapa.end()){
        cout << mapa[2] << endl;
    } else {
        cout << "Chave não encontrada!" << endl;
    }
}
```

Percorrer

Podemos percorrer os elementos do `map` utilizando algumas artimanhas da linguagem:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    map<int, string> mapa;
    mapa[1] = "um";
```

```
mapa[2] = "dois";  
mapa[3] = "três";  
  
for (auto [chave, valor] : mapa) {  
    cout << chave << ": " << valor << "\n";  
}  
}
```

Complexidade

A estrutura `map` é implementado com uma estrutura de Ávore Rubro-Negra (estudadas em ED e EDA), o que permite que todas essas operações de busca, inserção e remoção tenham complexidade $O(\log N)$.

Há também o `unordered_map` que implementa essas operações utilizando uma função hash (estudado em EDA), que permitem que essas operações sejam feitas em uma complexidade média de $O(1)$.

Operação	<code>map</code>	<code>unordered_map</code>
Busca	$O(\log N)$	$O(1)$
Inserção	$O(\log N)$	$O(1)$
Remoção	$O(\log N)$	Médio: $O(1)$
Percorrer	$O(N)$ (percorre ordenado)	$O(N)$
Menor / Maior elemento	$O(1)$	$O(N)$
Busca por ordem (<code>lower_bound</code> , <code>upper_bound</code>)	$O(\log N)$	Não tem suporte

No pior caso, ao se inserir chaves bem específicas que causem conflitos no `unordered_map`, essas operações podem ser realizadas em complexidade $O(n)$. A probabilidade de isso ocorrer geralmente é pequena, mas alguns juizes online como `Codeforces` podem trazer casos de testes que "hackeiam" essa estrutura. Nesse caso, podemos usar o `map` ou criar funções hash customizadas.